

3주차 예습과제

[4장] 분류

- 분류의 개요
- 결정 트리
- 앙상블 학습
- 랜덤 포레스트

1) 분류의 개요

분류(Classification): 지도학습의 대표 유형으로 기존 데이터가 어떤 레이블에 속하는지 패턴을 알고리즘으로 인지한 후 새로 관측된 데이터에 대한 레이블을 판별한다

- 학습 데이터로 주어진 데이터의 피쳐와 레이블 값을 머신러닝 알고리즘으로 학습해 모델 생성
- 생성된 모델에 새로운 데이터 값이 주어지면 미지의 레이블 값을 예측

앙상블(Ensemble): 정형 데이터의 예측 분석 영역에서 매우 높은 예측 성능을 보여 사랑받는 알고리즘

일반적 분류: 배깅(Bagging), 부스팅(Boosting)

랜덤 포레스트는 대표적인 '배깅' 방식이며, 근래의 앙상블 방법은 부스팅 방식으로 지속해서 발전 중

앙상블은 대부분 동일한 알고리즘을 결합하고, 기본 알고리즘으로는 결정 트리를 사용한다

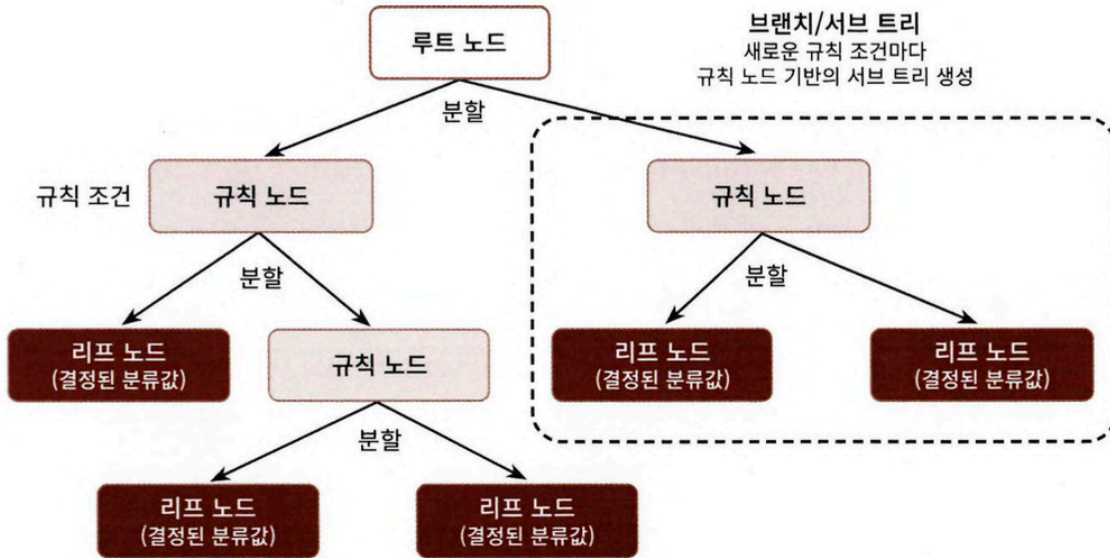
결정 트리: 쉽고 유연하게 적용되는 알고리즘으로 데이터의 사전 가공(스케일링, 정규화)의 영향이 적다

+)예측 성능을 향상시키기 위해 복잡한 규칙을 가져서 과적합이 발생해 오히려 성능이 떨어질 수도 있는데, 이 점이 앙상블 기법에서는 오히려 장점이 될 수 있다

2) 결정 트리

데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 tree 기반의 분류 규칙을 만드는 것

- if/else 기반으로 가장 쉽게 규칙을 표현할 수 있음
- 데이터의 어떤 기준을 바탕으로 규칙을 만들어야 가장 효율적인 분류가 될 것인지가 알고리즘의 성능을 좌우함.



피처가 결합해 규칙 조건을 만들 때마다 규칙 노드가 생성+많은 규칙 존재(= 분류 결정하는 방식이 더욱 복잡해짐) → 과적합

즉, 트리의 깊이가 깊어질수록 결정 트리의 예측 성능이 저하될 가능성이 높다

최소한의 결정 노드로 높은 예측 정확도를 가지려면, 데이터를 분류할 때 최대한 많은 데이터 세트가 해당 분류에 속할 수 있도록 결정 노드의 규칙이 정해져야 한다

‘어떻게 트리를 분할할 것인가’가 핵심 - 최대한 데이터 세트를 균일하게 구성하도록 분할하는 것이 필요

결정 노드는 정보 균일도가 높은 데이터 세트를 먼저 선택하도록 규칙 조건을 만들 (균일도가 높아야 예측이 쉬움)

조건에 맞는 서브 데이터 세트 만들 → 그 서브 데이터 세트에서 균일도 높은 데이터 세트를 쪼갬 → 계속 반복하며 내려가는 방식으로 데이터 값을 예측

정보의 균일도를 측정하는 방법: 정보 이득 지수(엔트로피 이용)와 지니 계수(불평등과 관련, 0이 가장 평등 → 1)

결정 트리 알고리즘을 사이킷런에서 구현한 DecisionTreeClassifier는 기본으로 지니 계수를 이용해 데이터 세트를 분할한다

데이터 세트를 분할하는 데 가장 좋은 조건(정보 이득 높 or 지니 계수 낮)을 찾아서, 자식 트리 노드에 걸쳐 반복적으로 분할한 뒤, 데이터가 모두 특정 분류에 속하게 되면 분할을 멈추고 분류를 결정한다

결정 트리 모델의 특징

[장점]

- 정보의 균일도를 규칙으로 하고 있어 알고리즘이 쉽고 직관적
- 규칙/리프 노드가 어떻게 만들어지는지 알 수 있고, 시각화 가능
- 균일도 외 고려사항이 없어 전처 작업(피처의 스케일링, 정규화) 불필요

[단점]

- 과적합으로 알고리즘 정확도 떨어짐
- 피처가 많고 균일도가 다양할수록 트리의 깊이가 커지고 복잡해짐(트리의 크기를 사전에 제한하는 튜닝이 필요)

결정 트리 파라미터

사이킷런에서 제공하는 결정 트리 알고리즘 클래스: `DecisionTreeClassifier` (분류), `DecisionTreeRegressor` (회귀)

파라미터 (둘다 동일)

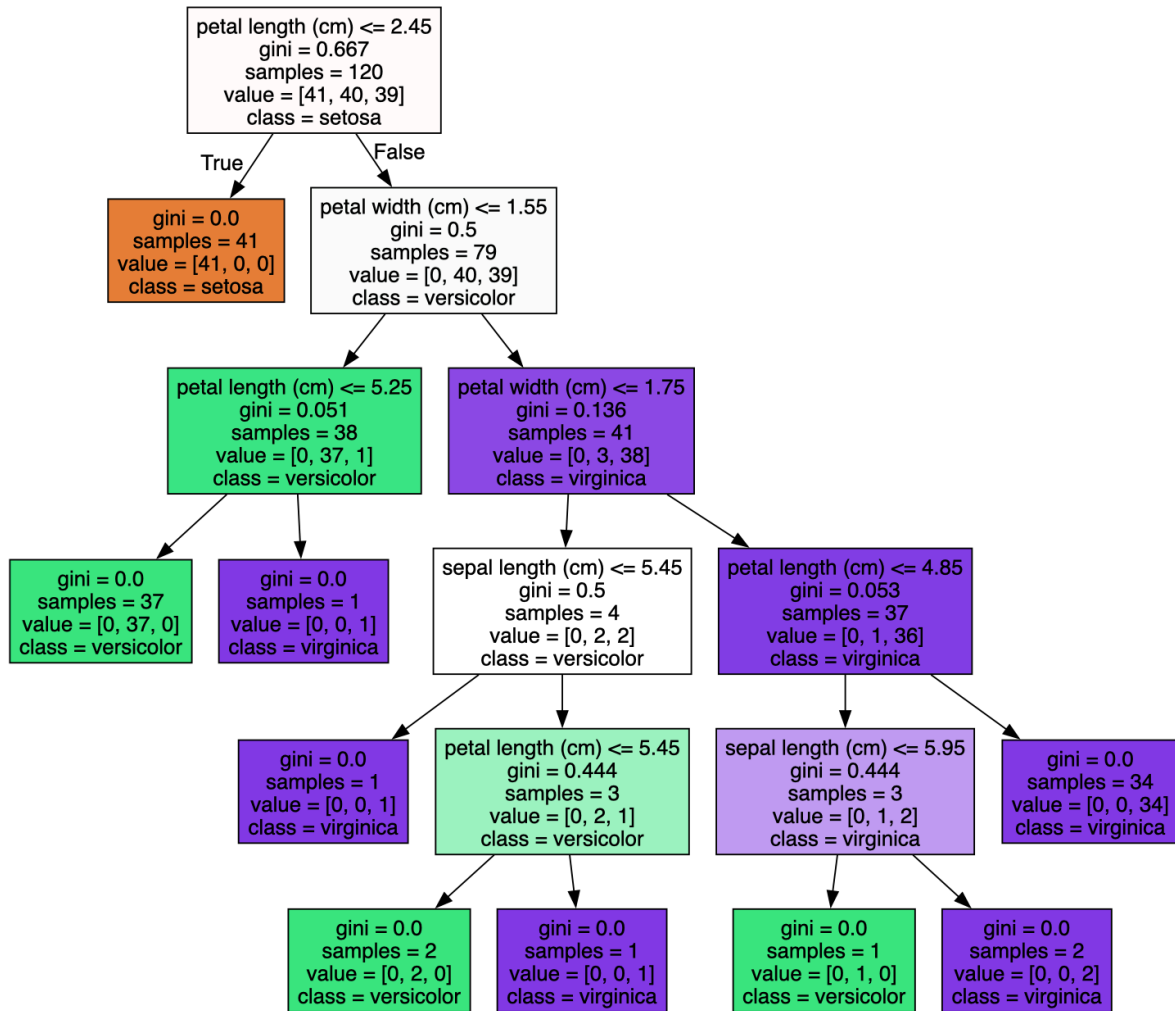
- `min_samples_split`: 노드를 분할하기 위한 최소한의 샘플 데이터 수, 과적합 제어용 (디폴트=2, 작게 설정할수록 분할 노드가 많아져 과적합 가능성 증가)
- `min_samples_leaf`: 분할될 경우 좌우 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수, 과적합 제어용(비대칭적 데이터의 경우에는 특정 클래스의 데이터가 극도로 작을 수 있으므로 이 경우 작게 설정해야 함)
- `max_features`: 최적의 분할을 위해 고려할 최대 피처 개수(디폴트=None, 데이터 세트의 모든 피처 사용해 분할)
- `max_depth`: 트리의 최대 깊이를 규정. (디폴트 None) `max_leaf_nodes`: 말단 노드 (Leaf)의 최대 개수

결정 트리 모델의 시각화

Graphviz 패키지를 사용하여 결정 트리 알고리즘이 어떤 규칙으로 트리를 생성하는지 시각적으로 확인 가능

사이킷런에서 제공하는 `export_graphviz()` API를 활용해, 함수 인자로 학습이 완료된 Estimator, 피처의 이름 리스트, 레이블 이름 리스트를 입력하면 학습된 결정 트리 규칙을 실제 트리 형태로 시각화해 볼 수 있다.

[11]:



결정 트리 규칙 구성

자식 노드가 없는 노드-리프 노드

- 최종 클래스(레이블) 값이 결정되는 노드
- 하나의 클래스 값으로 최종 데이터 구성되거나, 리프 노드가 될 수 있는 하이퍼 파라미터 조건을 충족하면 됨

자식 노드가 있는 노드-브랜치 노드

- 자식 노드를 만들기 위한 분할 규칙 조건을 가짐

노드 내 기술된 지표의 의미

- $\text{petal length} \leq 2.45 \dots$ 같은 피쳐 조건: 자식 노드 만들기 위한 규칙 조건. 이게 없으면 리프 노드

- gini: value=[]로 주어진 데이터 분포에서의 지니 계수
- samples: 현 규칙에 해당하는 데이터의 건수
- value = []: 클래스 값 기반의 데이터 건수.

하이퍼 파라미터: 복잡한 트리 생성 방지를 위한 용도로 노드가 분할되기 위한 조건을 설정함으로써 결정 트리를 간결하게할 수 있다

feature_importances: 피처가 트리 분할 시 정보이득/지니계수를 얼마나 효율적으로 잘 개선시켰는지 정규화된 값으로 표현한 것으로, 일반적으로 값이 높을수록 피처의 중요도가 높다

3) 앙상블 학습

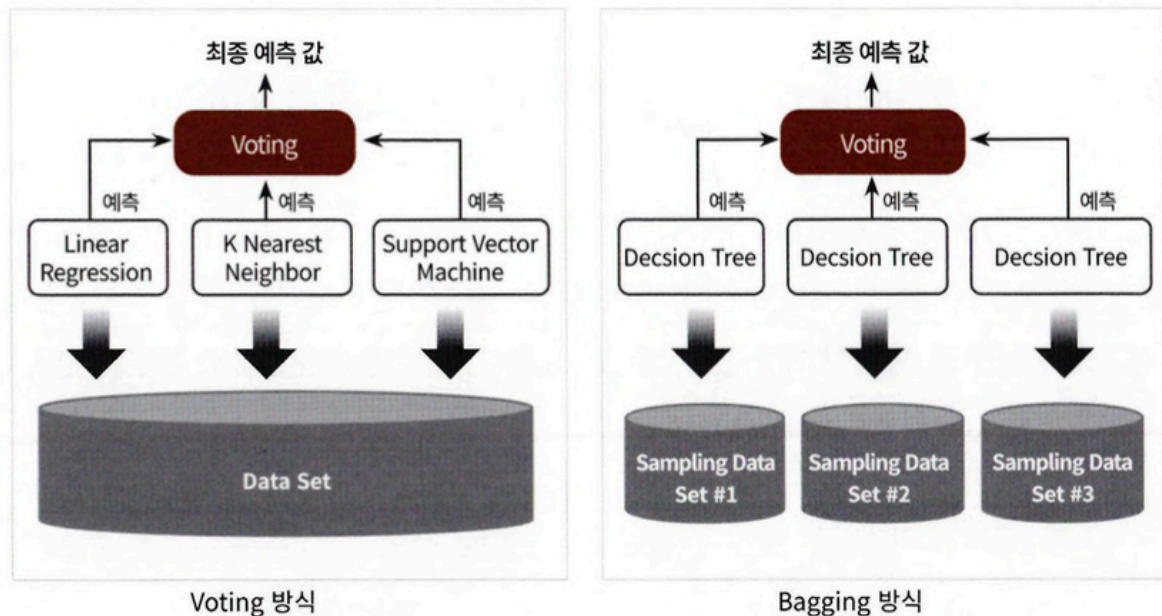
앙상블 학습 개요

앙상블 학습을 통한 분류: 여러 개의 분류기(Classifier)을 생성하고, 그 예측을 결합함으로써 정확한 최종 예측을 도출하는 기법

- 정형 데이터 분류에 뛰어난 성능을 나타냄
- 대표적인 앙상블 알고리즘: Random forest, Gradient Boosting

앙상블 학습의 유형

- 보팅(Voting): 여러 개의 분류기가 투표를 통해 예측 결과를 결정하는 방식, 서로 다른 알고리즘을 가진 분류기를 결합한다
- 배깅(Bagging): 기본 방식은 보팅과 동일하지만 각각의 분류기가 같은 유형의 알고리즘을 기반으로 하지만, 데이터 샘플링을 다르게 가져가며 학습을 수행해 보팅 수행한다 (대표: 랜덤 포레스트 알고리즘)
- 부스팅(Boosting) 등



왼쪽: 보팅 분류기-3개의 다른 머신러닝 알고리즘이 동일한 데이터셋에 대해 학습하고 예측한 결과를 가지고 보팅을 통해 최종 예측 결과를 선정

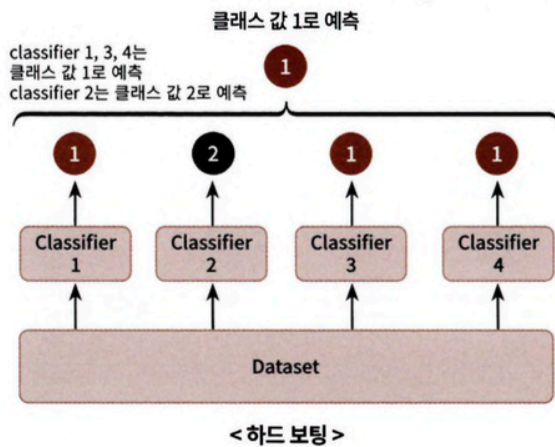
오른쪽: 배깅 분류기-단일 머신러닝 알고리즘으로 여러 분류기가 학습으로 개별 예측

- **부트스트래핑 bootstrapping 분할 방식:** 개별 분류기에 할당된 학습 데이터는 원본 학습 데이터를 샘플링해 추출하는 방식
- **배깅 앙상블 방식:** 개별 분류기가 이 방식으로 샘플링된 데이터셋에 대해 학습을 통해 개별 예측을 수행한 결과를 보팅을 통해 최종 예측 결과를 선정하는 방식

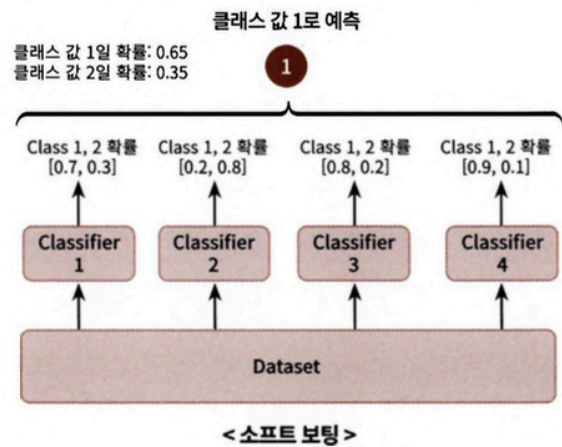
부스팅: 여러 개의 분류기가 순차적으로 학습 수행하되, 앞에서 예측이 틀린 데이터에 대해서는 다음 분류기에 가중치를 부여하면서 학습과 예측을 진행하는 방식 (계속해서 가중치를 부스팅하며 학습을 진행)

보팅 유형-하드 보팅, 소프트 보팅

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



하드 보팅을 이용한 분류: 다수결 원칙과 유사하며 예측 결과값 중 다수의 분류기가 결정한 예측값을 최종 보팅 결과로 선정한다

소프트 보팅: 분류기들의 레이블 값 결정 확률을 모두 더하고 평균내어 가장 확률이 높은 레이블 값을 최종 보팅 결과값으로 선정한다(일반적으로 예측 성능이 더 좋아서 더 많이 사용됨)

보팅 분류기

VotingClassifier : 사이킷런에서 제공하는 보팅 방식의 앙상블 구현 클래스로, 주요 생성 인자로 estimators와 voting 값을 입력받음

- estimators: 리스트 값으로 보팅에 사용될 여러 개의 classifier 객체를 튜플 형태로 입력받음
- voting: hard(기본) / soft

```
# 개별 모델은 로지스틱 회귀와 KNN임.
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)

# 개별 모델을 소프트 보팅 기반의 앙상블 모델로 구현한 분류기
vo_clf = VotingClassifier( estimators=[('LR', lr_clf), ('KNN', knn_clf)], voting='soft' )

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2, random_state= 156)

# VotingClassifier 학습/예측/평가.
vo_clf.fit(X_train, y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```



```
# 개별 모델의 학습/예측/평가.
classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train, y_train)
    pred = classifier.predict(X_test)
    class_name= classifier.__class__.__name__
    print('{0} 정확도: {1:.4f}'.format(class_name, accuracy_score(y_test, pred)))
```

[Output]

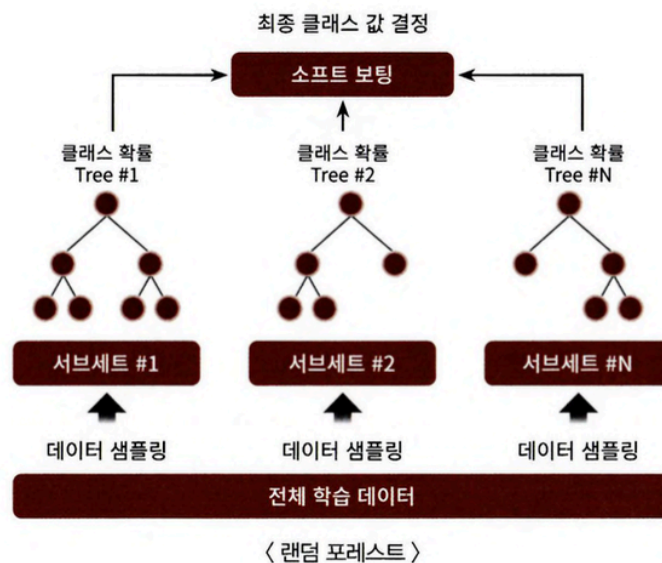
```
Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

3) 랜덤 포레스트

랜덤 포레스트의 개요 및 실습

배깅의 대표적인 알고리즘 중 하나이다 (배깅: 같은 알고리즘으로 여러 분류기를 만들어 보팅으로 최종 결정하는 알고리즘)

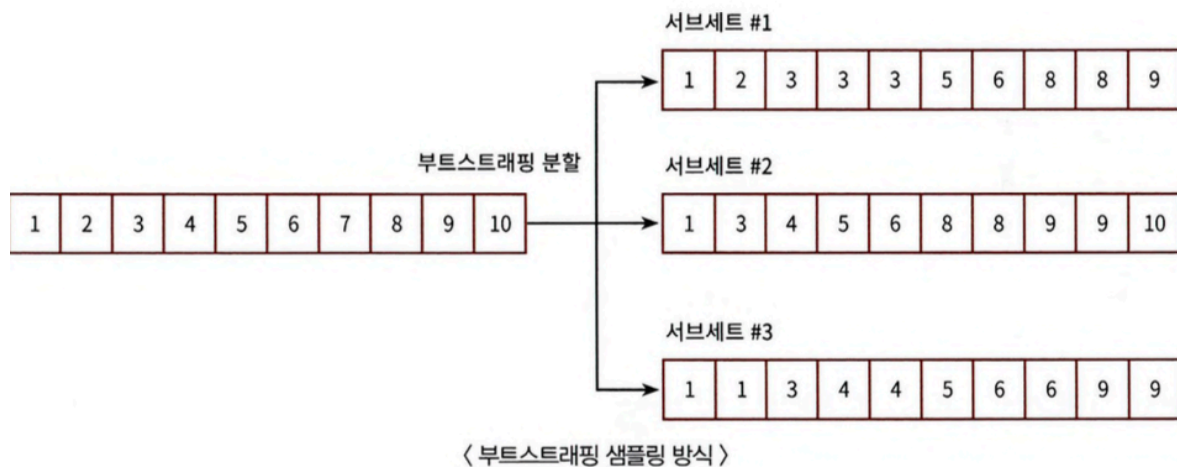
- 빠른 수행 속도
- 여러 영역에서 높은 성능 예측
- 기반 알고리즘: 결정 트리-여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링해 개별 학습을 수행한 뒤, 최종적으로 모든 분류기가 보팅을 통해 예측 결정



랜덤 포레스트는 개별 분류기의 기반 알고리즘은 결정 트리지만, 개별 트리가 학습하는 데이터셋은 전체 데이터에서 일부가 중복되게 샘플링된 데이터셋이다

부트스트래핑: 위와 같이 여러 개의 데이터셋이 중복되도록 분리하는 방식

→ 랜덤 포레스트의 subset 데이터는 부트스트래핑으로 데이터가 임의 생성된다.



위와 같이 데이터가 중복된 개별 데이터셋에 결정 트리 분류기를 각각 적용 하는 게 랜덤 포레스트이다

`RandomForestClassifier` : 사이킷런이 지원하는 랜덤 포레스트 기반 분류 클래스

```
# 랜덤 포레스트 학습 및 별도의 테스트 셋으로 예측 성능 평가
rf_clf = RandomForestClassifier(random_state=0)
rf_clf.fit(X_train , y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test , pred)
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy))
```

랜덤 포레스트 하이퍼 파라미터 및 튜닝

파라미터(하이퍼파라미터와 대부분 동일)

- `n_estimators`: 결정 트리의 개수 지정한다(디폴트=10개) 많이 설정할수록 성능이 좋아 지지만 계속 증가시킨다고 무조건 향상되는 건 아니고, 늘릴수록 학습 수행 시간이 늘어난다
- `max_features`: 결정 트리에서 쓰인 것과 동일하지만 여기서 디폴트는=auto (결정트리는 None)

- max_depth, min_samples_leaf, min_samples_split: 결정트리에서의 사용 방식과 동일하다